🎤 1 Minute — Game Engine (Technical Explanation + How It Links Everything)

After the Player and AI classes, the core component of our system is the GameEngine. While players define what actions are possible, the GameEngine controls the whole game process.

When the game starts, it creates all players, shuffles the deck, and deals four cards each. From there, it manages the turn order, tracks used actions, and ensures rules like mandatory discard are followed.

All four actions go through this engine — draw cards, draw-and-discard, steal, and discard groups. The engine validates every move, updates the card state, and checks for a win immediately.

Once the player ends their turn, the engine switches to AI automatically, triggers their decision logic, performs their move, and then returns control back to the player. This creates a continuous gameplay loop. In short, the GameEngine is the brain of our game, coordinating actions, rules, and turn flow."

🎤 1 MIN — Technical Explanation of the Game Engine (How it works as a system)

"The Game Engine is the core controller of Notty. It handles everything behind the scenes — player creation, turn order, card distribution, actions, and win detection. When the game starts, the engine builds a deck, shuffles it, creates the human player and AI opponents, and deals four cards to each player.

During gameplay, the engine maintains the state through variables such as used_draw, used_steal, used_draw_discard and pending_discard_required. These flags ensure that players follow the rules — for example, drawing up to three cards a turn, or enforcing that draw-one-discard-one must finish before ending the turn.

All four actions are implemented inside the engine: drawing, draw-and-discard, stealing, and discarding groups. Group validation checks rules like sequences or same-rank sets. After every move, the engine immediately checks for win conditions. When a turn ends, it resets all flags and shifts the turn to the next player.

The engine also controls the AI phase. When it's the AI's turn, it calls ai_take_turn to generate an action using decision logic, updates the pile or hand, then moves to the next player automatically. This creates a full gameplay loop where players and AI interact seamlessly. Essentially, the engine links deck, players, UI, and rules together and drives the entire game."

🎤 1 MIN — My Personal Contribution & Development Process

My main responsibility was building the GameEngine, which controls the entire game flow — initialization, player and AI turns, action validation, and win detection. I implemented deck management, turn-switching, action flags like used_draw and pending_discard_required, and the two-phase draw-one-discard-one flow. I also handled group discard validation and integrated AI decisions, ensuring smooth automatic turns and edge cases like reshuffling when the deck runs out.

For the interface, I added a winner pop-up with a darkened, blurred background. When the game ends, it displays an animated victory message and sound, and clicking anywhere restarts the game.

In short, I built the game's core logic and added a simple interactive visual for game completion, connecting the GameEngine to the interface in a clear and engaging way.

🎤 1 MIN — My Personal Contribution & Development Process

My main responsibility in this project was building the GameEngine. I started by planning the overall game loop — initialization, player turns, AI turns, win detection, and action validation. I built the deck management, dealing logic, and turn-switching first. Then I introduced action flags like used_draw, used_steal, and a pending_discard_required state to prevent illegal moves.

One of the more challenging parts was implementing draw-one-discard-one without breaking turn logic. I solved this using a two-phase flow where the player is locked until a discard is completed. Next, I implemented logic to verify group validity for discarding — either sequences or same-rank sets.

I also integrated the AI with this engine, so AI decisions can be triggered automatically during their turn. Throughout development I tested each component by simulation, resolving edge cases like running out of cards using automatic reshuffling. By completing the GameEngine, I essentially built the foundation that connects all modules and makes the game playable end-to-end."

For the interface, I contributed a winner pop-up with a darkened, blurred background. When the game ends, this overlay appears with an animated victory message, and clicking anywhere on it restarts the game.

In short, I built the brain of the game while adding a simple, interactive visual feedback for game completion, connecting the GameEngine logic to the interface in a clear and engaging way.

Q1: What was the main challenge in building the interface?

The most challenging part was connecting UI actions to the GameEngine while keeping the state updated correctly. I had to make sure when a player clicks draw, steal, or discard, the UI triggers the logic, receives the updated state, and instantly re-renders cards. Handling card selection and draw–discard interaction also required careful event management. After refining the structure and adding animation feedback, the interface became smooth and responsive.

Q2: How does your UI communicate with the GameEngine?

Every button click sends a request to the GameEngine, for example draw or steal. The engine processes the logic, updates hands, and returns results. The UI then reads the new game state and renders it visually — cards, turn indicator, discard pile, AI moves, and win status. This creates a loop where UI handles interaction and GameEngine handles logic.

Q3: What did you improve to enhance user experience?

I added hover highlights, smooth card animations, sound effects, and clear turn indicators. When actions happen — draw, steal or group discard — the UI visually shows cards moving so players understand the result instantly. This made gameplay more intuitive and engaging, not just functional.

Q4: Why is your part important to the whole system?

The UI is the part the player actually sees and interacts with. Without it, the game logic cannot be played. My interface transforms the GameEngine into a usable product — visual cards, buttons, animations, and feedback — making the game playable and user-friendly.

Q5: What did you learn from this task?

I learned how to structure UI and game logic cleanly, manage states, and sync visual changes with backend updates. It improved my understanding of event handling, animations, and user-centered design. Overall it strengthened both my coding and game development skills.

Q1: Main challenge in the interface?
Connecting UI actions to the GameEngine and keeping card states updated smoothly.

Q2: How does UI communicate with GameEngine?
Buttons send actions to the engine, engine updates logic, UI refreshes visuals instantly.

Q3: What improvements enhance UX?
Added hover highlights, card animations, sounds, and clear turn indicators.

Q4: Why is your part important?
It turns the game logic into a playable, visual, and interactive experience.

Q5: What did you learn?
How to sync UI with logic, handle events, animations, and improve user experience.

Q6: How do you handle game-over?

Show a popup with winner message, avatar, and play victory sound.

Q7: How does card stealing animation work?

Player clicks NPC → engine updates → UI animates card flying from NPC to player.

Q8: How do you manage multiple card layouts?

Snapshots from engine define positions; layout functions arrange player, AI, deck, and discard cards

1. Overall design and responsibility

Q: What is the main role of the GameEngine in your project?
A: It controls the full game flow, including turns, actions, AI decisions, win detection, deck and discard management, and connecting the game logic to the UI.

2. Player action logic

Q: How did you implement "draw-one-discard-one" safely?
A: Using a two-phase system with a pending_discard_required flag. The player draws a card first, then must discard one before ending the turn, preventing illegal moves.

Q: How do you ensure a player cannot perform the same action multiple times?
A: I used flags like used_draw, used_steal, and used_draw_discard to track actions per turn.

3. Group validation for discards

Q: How does the engine check if a group of cards is valid?
A: Two methods: _is_sequence_group checks consecutive numbers of the same suit; _is_same_rank_diff_suit_group checks same rank with different suits. Only valid groups can be discarded.

4. AI integration

Q: How is AI implemented?
A: AI uses think_action to decide moves, and ai_take_turn executes them. The engine manages automatic AI turns sequentially and checks win conditions after each action.

Q: Can AI win mid-turn?
A: Yes, the engine immediately checks ai.has_empty_hand() after each action and sets self.winner if AI has no cards.

5. Turn management

Q: How does the engine handle player and AI turns?
A: current_index tracks the current player. next_player() loops through all players. end_player_turn() ensures the player completed mandatory actions before moving to the next turn.

6. Deck & discard pile handling

Q: How do you handle the deck running out?
A: recycle_discard_pile() moves the discard pile back into the deck and shuffles it when the deck is empty.

7. Game reset

Q: How does reset_game() work?
A: It reinitializes the deck, discard pile, players' hands, current index, and winner, and then redeals initial cards.

8. UI integration

Q: How does the GameEngine connect to the UI?
A: Methods like ui_snapshot() provide all data (player hands, AI hands, deck, discard pile, current player, winner) for UI rendering in real-time.

9. Difficulty setting

Q: How can the AI difficulty be changed during the game?
A: set_difficulty() recreates AI players based on the chosen difficulty and resets the game session.

10. Edge cases

Q: How do you handle invalid player actions?
A: Every action method returns a structured response (ok, reason, message) for errors like invalid discard index, full hand, or action already used.

GameEngine Rapid-Fire Q&A

1. **Q:** What is the GameEngine's main role?
   A: It controls the game flow, player/AI turns, deck management, action validation, and win detection.

2. **Q:** How do you handle a player's draw-one-discard-one action?
   A: Using pending_discard_required to force discard after draw before ending the turn.

3. **Q:** How do you prevent illegal actions?
   A: Flags like used_draw, used_steal, used_draw_discard track per-turn actions.

4. **Q:** How does the engine validate group discards?
   A: _is_sequence_group checks sequences; _is_same_rank_diff_suit_group checks same rank with different suits.

5. • **Q:** How is AI integrated?
   A: AI decides via `think_action`, then `ai_take_turn` executes, and engine checks for win after each action.
6. • **Q:** Can AI win mid-turn?
   A: Yes, `ai_take_turn` sets `self.winner` if AI's hand is empty immediately.
7. • **Q:** How are turns managed?
   A: `current_index` tracks player; `next_player()` loops; `end_player_turn()` ensures required actions are completed.
8. • **Q:** What happens if the deck is empty?
   A: `recycle_discard_pile()` reshuffles the discard pile back into the deck.
9. • **Q:** How does `reset_game()` work?
   A: Resets deck, discard pile, players' hands, current player, winner, and redeals initial cards.
10. • **Q:** How is the engine connected to the UI?
    A: `ui_snapshot()` provides hands, deck, discard pile, current player, and winner for display.
11. • **Q:** How can AI difficulty change?
    A: `set_difficulty()` rebuilds AI players and restarts the game session.
12. • **Q:** How are invalid actions handled?
    A: Methods return structured responses with `ok`, `reason`, and `message`.
13. • **Q:** How are rounds counted?
    A: `round` increments after all players have taken turns and the index returns to the human player.
14. • **Q:** How are multiple AI turns handled?
    A: `run_ai_phase()` loops AI players sequentially, executing actions until it returns to the human player.
15. **Q:** How is draw-and-discard different from draw-up-to-three?
    A: Draw-and-discard is forced in two phases with one discard; draw-up-to-three allows up to three draws per turn.

GameEngine Quick Cheat Sheet

Main Role

- Controls game flow: player/AI turns, deck, actions, win detection.

Player Actions

- **Draw-One-Discard-One:** pending_discard_required enforces discard before ending turn.
- **Draw-Up-To-Three:** used_draw limits draws per turn.
- **Steal:** used_steal prevents repeated stealing.
- **Discard Group:** _is_sequence_group for sequences, _is_same_rank_diff_suit_group for same rank/diff suits.

AI Integration

- AI decides via think_action.
- ai_take_turn() executes action, updates winner if hand empty.
- run_ai_phase() loops all AI turns until human's turn.
- Difficulty set via set_difficulty() rebuilds AI players.

Turn Management

- current_index tracks turn; next_player() loops.
- end_player_turn() ensures required actions done.
- round increments when human turn starts again.

Deck & Discard

- recycle_discard_pile() shuffles discard back into deck if empty.
- reset_game() resets deck, discard pile, hands, winner, redeals initial cards.

UI Connection

- ui_snapshot() gives hands, deck, discard, current player, winner.

Action Validation

- Invalid actions return ok=False with reason and message.
- Flags prevent illegal moves (used_draw, used_steal, used_draw_discard).

Miscellaneous

- Draw-and-discard = forced two-phase action.
- Draw-up-to-three = optional up to 3 draws.
- AI can win mid-turn; engine updates winner instantly.